AtlasScout

Architecture & CLI Specification Document

This document outlines the standard Command Line Interface (CLI), internal data structures, and standard output formats for the **AtlasScout** sprite localization tool. It incorporates the standard UNIX philosophy of utilizing `stdout` strictly for data piping when operating in quiet mode.

1. Command Line Interface (CLI) Flags

The CLI is designed to be highly composable. Standard output (JSON/TSV) is printed to `stdout`, while progress messages and errors should be written to `stderr`. Using the `-q` flag ensures that only parseable data hits `stdout`.

Flag	Long Form	Description
`-c`	--config <file>	Path to a JSON configuration file. Overrides individual CLI flags.
`-s`	--sprites <path>	Path, directory, or pattern for the source sprites to search for.
`-a`	--atlases <path>	Path, directory, or pattern for the atlases (spritesheets) to scan.
`-r`	--recurse	Recursively search subdirectories for both source sprites and atlases.
`-o`	--output <file>	Output file path. If omitted or set to `-`, defaults to `stdout`.
`-f`	--format <type>	Built-in output format. Options: `json` (default), `tsv`, `csv`.
`-t`	--type <name>	Force a specific search `.so` plugin implementation (e.g., `avx2`).
`-l`	--list-types	List available search types (dynamically loaded from plugins) and exit.
`-q`	--quiet	Suppress all progress/status messages (silences `stderr`). Ideal for piping.
`-h`	--help	Print usage information and exit.

> **Unix Pipeline Best Practice:** By routing all progress bars, warnings, and informational messages to `stderr`, you guarantee that running `atlasscout -s ./sprites -a ./atlas.png -f tsv -q | awk '{print $1, $4, $5}'` will execute flawlessly without parsing errors.

2. Internal Data Structure

Once a match is found using the chosen search plugin, the data is stored in the following memory-efficient C-struct before being passed to the selected built-in output formatter. As specified, the path refers to the source sprite file.

```
struct SpriteMatch {
    char sprite_name[256];       // E.g., "player_run_01"
    char sprite_path[1024];      // Path to the source sprite file searched
    char spritesheet_name[256];  // Name of the atlas it was found inside

    // Spatial data within the spritesheet
    int  x;
    int  y;
    int  width;
    int  height;

    int  is_rotated;             // 1 if rotated 90 degrees, 0 otherwise
};
```

3. Standard JSON Output Format

When `-f json` is used (or no format is specified), the application outputs a structured payload. This is designed to be easily consumed by game engines, Python automation scripts, or web tools.

```
{
  "matches": [
    {
      "sprite_name": "player_run_01",
      "sprite_path": "./source_assets/characters/player_run_01.png",
      "atlas_name": "main_texture_atlas_v2",
      "bounds": {
        "x": 1024,
        "y": 256,
        "width": 64,
        "height": 128
      },
      "rotated": false
    }
  ],
  "metadata": {
    "total_found": 1,
    "search_plugin_used": "avx2_simd"
  }
}
```

4. TSV Format (Pipeline Example)

When `-f tsv` is specified, the application will output tab-separated values. This is strictly flat data, making it perfect for shell tools like `awk`, `cut`, or `grep`.

```
# Output format:
# SPRITE_NAME    SPRITE_PATH     ATLAS_NAME      X       Y       W       H       ROTATED
player_run_01    ./source_assets/characters/player_run_01.png    main_texture_atlas_v2    1024    256     64      128     0
```